

GPT-SoVITS

Two stages, one vocabulary: a language model that speaks through a VITS vocoder

A conceptual walkthrough of GPT-SoVITS v2 — grounded in the `voice` pure-Rust reimplementation

Abstract. — GPT-SoVITS turns **text** into **speech in a particular voice**, given only a few seconds of that voice as a reference. It does so by splitting the problem in two along a boundary that is worth the whole document: a discrete vocabulary of **semantic tokens**, twenty-five per second, drawn from a codebook of 1024 entries. One stage — `s1`, the “GPT” half — is an autoregressive language model that predicts that token sequence from phonemes. The other — `s2`, the “SoVITS” half — is a VITS-lineage conditional variational autoencoder wrapped around a HiFi-GAN vocoder, which renders those tokens as a waveform in the reference speaker’s timbre. This document explains why that split is the right one, what each half is made of, how both are trained, and — because a port is only as good as its checks — how the implementation is verified beyond counting loaded weights.

Contents

1	Why two stages	2
2	Background: a few building blocks	3
3	The semantic-token boundary	4
	3.1 cnhubert — a self-supervised speech representation	4
	3.2 25 Hz: one stride decides everything downstream	4
	3.3 The codebook — 1024 entries and a nearest-neighbour lookup	5
4	<code>s1</code> — text to semantic tokens	6
	4.1 Generation by continuation	6
	4.2 The block attention mask	6
	4.3 One token at a time: the KV cache	7
	4.4 Choosing the next token	7
5	<code>s2</code> — a conditional VAE around a vocoder	7
	5.1 The prior: <code>enc_p</code> and the MRTE	8
	5.2 The posterior and the flow	9
	5.3 The speaker vector: <code>ref_enc</code>	9
	5.4 The vocoder: <code>dec</code>	10
	5.5 A note on dimensions (v2, 32 kHz)	10
6	Prosody: the frozen Chinese BERT	11
7	The phoneme table is a compatibility contract	11
8	Training I: <code>s1</code> , one loss that means something	11
9	Training II: <code>s2</code> , the VITS adversarial recipe	13
	9.1 The training forward	13
	9.2 The five terms	14
	9.3 Multi-period plus multi-scale	14
	9.4 Reading the loss curves	15

10	Verifying a port beyond weight coverage	15
11	Inference recap: the reverse path	16

1 Why two stages

Text-to-speech has to invent everything: which sounds occur, how long each one lasts, where the emphasis and the breaths fall, and what vocal tract is producing them. Trying to learn all of that with one network conflates two very different kinds of uncertainty. **What is said and how it is paced** is a long-range, discrete, highly structured decision — the same territory as language modelling. **What it sounds like** is a short-range, continuous, high-bandwidth signal-processing problem — the territory of vocoders.

GPT-SoVITS declines to conflate them. It defines an intermediate representation that is coarse enough for a language model to predict token by token, yet rich enough for a vocoder to render, and gives each half exactly one job:

- s1** — **what is said and how it is paced** a decoder-only transformer (**T2s**) that reads phonemes and emits a sequence of semantic tokens. It decides duration, rhythm and delivery implicitly, by choosing how many tokens each sound gets. There is no separate duration predictor and no alignment model — pacing is a by-product of generation.
- s2** — **what it sounds like** a conditional VAE (**SovitsPartial**) whose decoder is a HiFi-GAN vocoder. It reads the token sequence, the phonemes again, and a **speaker vector** extracted from the reference clip, and produces the waveform.

The seam between them is the subject of §3, and it is the single most important number in the system.

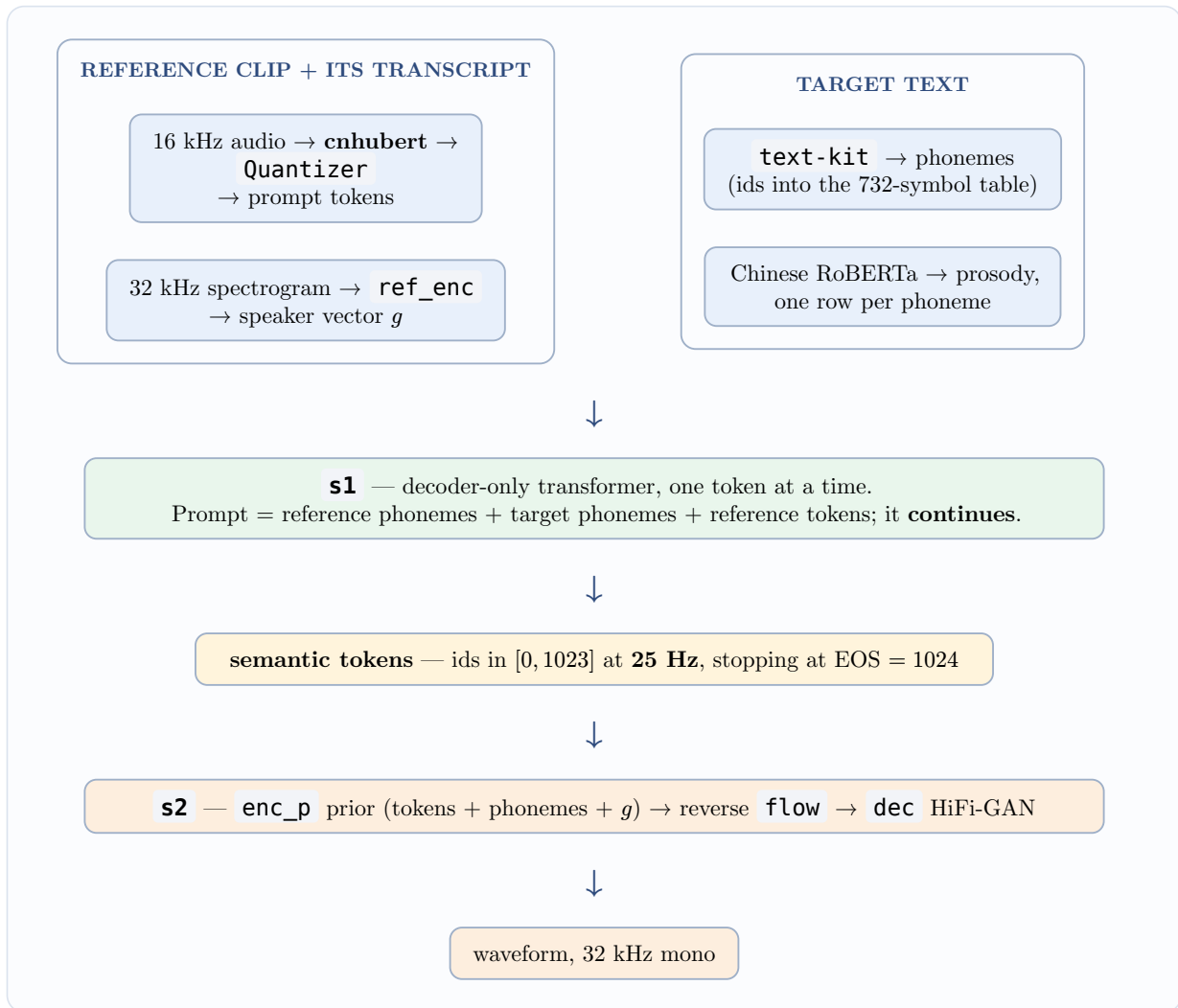


Figure 1: The synthesis path. The reference clip is used twice and for two different purposes — its **tokens** prime **s1**, its **spectrogram** becomes the speaker vector that colours **s2**. Everything in blue is a frozen front-end that is never trained here.

2 Background: a few building blocks

Readers new to speech synthesis may want these six ideas first; every later section leans on them. Experienced readers can skip ahead.

Phonemes and grapheme-to-phoneme a model is not shown letters but **phonemes** — the distinct sounds of a language. Turning writing into phonemes is **grapheme-to-phoneme** (g2p), and for Mandarin it means word segmentation, pinyin lookup, splitting each syllable into initial and final, and applying **tone sandhi** (tones that change in context). This is pure rule work, no learning, and it lives in **text-kit**.

Spectrogram (STFT) sliding a short window along a waveform and taking a **Fourier transform** of each window gives the **short-time Fourier transform** — a 2-D picture of how much energy sits at each frequency, moment by moment. Columns are **frames**; the number of samples between consecutive frames is the **hop**. A **mel spectrogram** regroups the frequency rows onto a perceptually spaced scale, so distance on it tracks “sounds alike”.

Self-supervised speech representations a model like HuBERT is trained on unlabelled audio to predict masked parts of its own input. It never sees a transcript, yet the vectors it produces encode phonetic content remarkably cleanly. Such a model is used **frozen**, as a universal analysis front-end (§3.1).

Vector quantisation a **codebook** is a fixed list of vectors. **Quantising** a continuous vector means replacing it with the index of the nearest codebook entry. That turns a continuous stream into a sequence of **discrete symbols** — which is exactly what a language model can be trained on (§3.3).

Autoregressive language models a decoder-only transformer predicts the next symbol from all the previous ones, and generates by feeding its own output back in. Two mechanisms recur below: an **attention mask**, which decides which positions may see which, and a **KV cache**, which stores the keys and values already computed so each new step costs one position rather than the whole sequence.

GANs and vocoders a **vocoder** turns a compact acoustic representation back into samples. Trained on a reconstruction loss alone it produces oversmoothed, buzzy audio, so a second network — the **discriminator** — is trained to tell real audio from generated, and the vocoder is trained to fool it. That adversarial pressure is what supplies crisp fine structure (§9).

3 The semantic-token boundary

Everything in this system is organised around one interface: **25 Hz token ids over a 1024-entry codebook**. `s1` predicts them; `s2` renders them; preparing a training corpus means running `Quantizer::encode` over it. Three components produce that interface, and they are worth taking in order.

3.1 cnhubert — a self-supervised speech representation

`cnhubert` is `TencentGameMate/chinese-hubert-base`, a stock HuBERT: 12 transformer layers, 768 wide, 95M parameters, consuming 16 kHz mono audio. It is loaded here as `Hubert`, mirroring the Hugging Face `state_dict` so the published checkpoint applies unchanged.

Its front-end is what sets the clock. Seven strided convolutions (`HubertConfig::conv_stride = [5, 2, 2, 2, 2, 2, 2]`) reduce the sample rate by their product:

$$5 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 320 \text{ samples per frame,} \quad \frac{16000}{320} = 50 \text{ frames per second.}$$

`HubertConfig::samples_per_frame` is that product, and a unit test pins it at 320 for precisely this reason.

The convolutions are followed by a projection to the model width and a **post-norm** transformer — normalisation **after** each residual addition, because this checkpoint sets `do_stable_layer_norm: false`. The distinction matters more than it looks: the pre-norm variant loads the same weights without complaint and computes something else. `Hubert::hidden_states` returns every layer's output, input embedding first, because callers sometimes want an intermediate layer rather than the last.

3.2 25 Hz: one stride decides everything downstream

`Quantizer` contains exactly two things: a strided convolution `ssl_proj` and a codebook. The convolution has kernel 2 and stride 2, so it halves the frame rate:

$$50 \text{ Hz} \rightarrow 25 \text{ Hz,} \quad \text{one token every } 40 \text{ ms.}$$

That single number propagates everywhere. A one-minute utterance is 1500 tokens, which is why `SynthOptions::max_tokens` defaults to 1500. `Clip::seconds` in the training corpus divides the token count by 25. And `s2`'s prior encoder wants 50 Hz features, so `SovitsPartial` repeats every token once on the way in — **nearest-neighbour** upsampling, never interpolation, because interpolating between two codebook indices does not name a third entry.

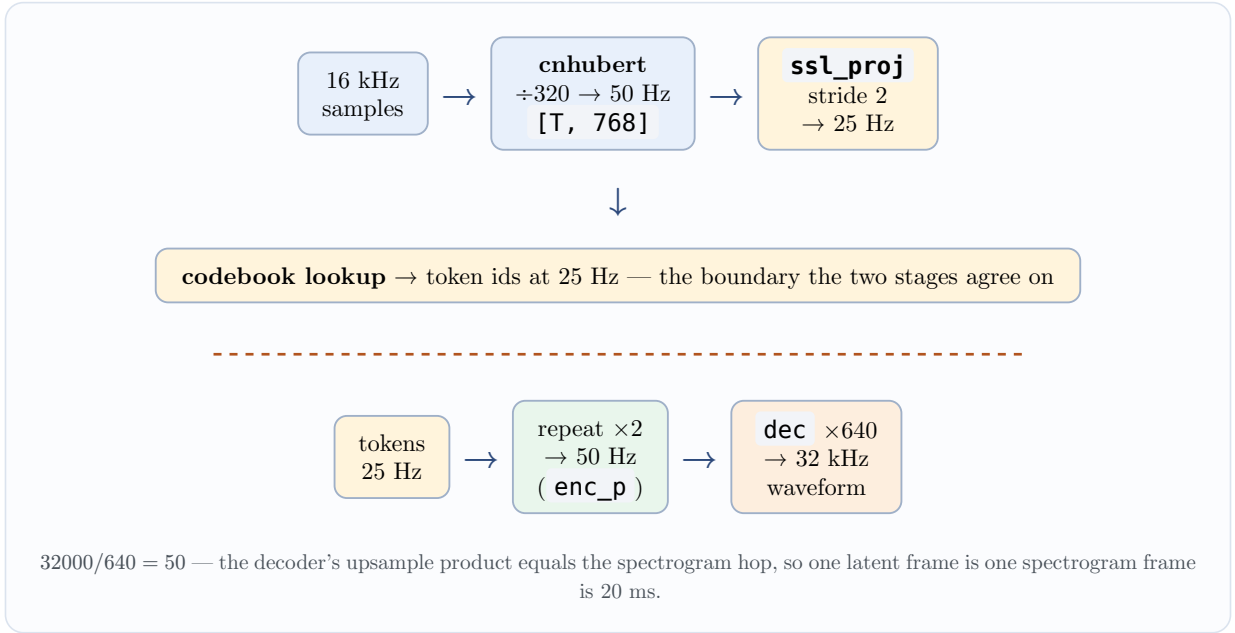


Figure 2: The rate chain, from samples to tokens and back. Every arrow is a fixed integer ratio; none of them is free to change independently.

3.3 The codebook — 1024 entries and a nearest-neighbour lookup

Codebook holds one parameter: `embed`, a 1024×768 matrix. Encoding a frame x_t is a nearest-neighbour search,

$$k_t = \operatorname{argmin}_j \|x_t - e_j\|_2^2,$$

and decoding is a row lookup, $\hat{x}_t = e_{k_t}$. The implementation expands the square,

$$\|x - e\|^2 = \|x\|^2 - 2x \cdot e^\top + \|e\|^2,$$

and **drops** $\|x\|^2$ because it is the same for every candidate j and so cannot change which one wins — leaving one matmul and one `argmax`. The correctness of that shortcut depends on keeping $\|e\|^2$, which is why the unit test uses a codebook where the nearest entry is deliberately **not** the one with the largest dot product.

Two consequences are worth stating plainly.

Why discretise at all. An autoregressive model needs a finite vocabulary to place a softmax over. Continuous acoustic frames give none; 1024 codebook entries give exactly one, so predicting speech becomes literally the same computation as predicting text. The codebook is the reason `s1` can exist.

Why 1024 and not more. The vocabulary has to be small enough that a model trained on modest data assigns useful probabilities to all of it, and coarse enough that the tokens carry **content and prosody** rather than **timbre** — timbre is supplied separately, by the speaker vector (§5.3). A codebook fine enough to capture voice identity would make `s1`'s job speaker-dependent, and the few-shot cloning that the whole design exists for would stop working.

The checkpoint also carries `embed_avg`, `cluster_size` and `inited` beside `embed`. These are the exponential-moving-average statistics used while **training** the codebook and have no role in a lookup, so they are not modelled and are reported unused — the 3 unused tensors in `s2`'s coverage report (§10).

4 s1 — text to semantic tokens

T2s is a decoder-only transformer: 24 post-norm layers, 512 wide, 16 heads, feed-forward width 2048. It embeds two vocabularies — 732 phonemes (`ar_text_embedding`) and 1025 semantic ids (`ar_audio_embedding`) — into the same 512-dimensional space, and `ar_predict_layer` projects back to 1025 logits.

The semantic vocabulary is 1025 rather than 1024 because of `T2sConfig::eos`, the **end-of-sequence** token: id 1024, the one id that is not a codebook entry. Generation stops when it is sampled. Off by one here and every utterance would end on a real token instead of stopping.

Prosody enters through `bert_proj`, which narrows the 1024-wide BERT features to 512 and **adds** them to the phoneme embeddings. Added, not concatenated: prosody **colours** each phoneme rather than lengthening the sequence, and a concatenation would double the prompt and silently misalign every position.

Positions are sinusoidal with one learnable scalar `alpha` per table (`SinePosition`) — the table is fixed, the scalar decides how loudly it speaks. Text and audio have **separate** position tables, and audio positions restart at zero, because upstream applies the two encodings before concatenating.

4.1 Generation by continuation

This is the part that is easy to get wrong and produces a bug that looks like a broken decoder.

s1 does not “read text and emit audio tokens”. It **continues a sequence**. The prompt it is given is:

1. the **reference clip’s** phonemes, followed by the **target text’s** phonemes — one text sequence; then
2. the **reference clip’s** semantic tokens — a real, coherent audio sequence that the first half of the text describes.

The model’s task is now unambiguous: the text ran on past where the audio stopped, so keep producing audio for the rest of it. Supply only the target text, and the model sees phonemes for one utterance beside audio of another, finds nothing to continue, and stops after a token or two. That is why `tts`’s `--reference-text` is **required** and not a nicety, and why `Reference` carries `phones` and `prosody` alongside its `tokens`.

4.2 The block attention mask

A plain causal triangle across the whole prompt would be wrong. The text is **given**, not predicted, so a phoneme has every right to see the phonemes after it; only the audio is generated and only the audio must be causal. `prompt_mask` therefore builds a **block** mask (`true` blocks):

	P_1	P_2	P_3	A_1	A_2	A_3
P_1	•	•	•			
P_2	•	•	•			
P_3	•	•	•			
A_1	•	•	•	•		
A_2	•	•	•	•	•	
A_3	•	•	•	•	•	•

rows = queries, columns = keys; P = phoneme positions, A = audio positions

Figure 3: The prompt mask, for 3 phonemes and 3 audio tokens. Shaded cells are allowed. Text is bidirectional within itself; audio sees all the text and its own past. A single causal triangle would blank the upper-right of the text block — weaker conditioning that nothing would report.

Once generation is under way each step has a single query, and the mask reduces to the ordinary causal one. `causal_mask` builds it with `Tensor::tril_mask` and an offset of $n_{kv} - n_q$ — **not** `triu_mask`. Burn names its mask helpers for the triangle they **keep**, so the obvious-looking choice reverses time, and when a step has one query and one key it masks the only position there is. A full row of $-\infty$ into a softmax is `NaN`, not an error, and it propagates to every logit. That exact bug shipped once already, in the Whisper port.

4.3 One token at a time: the KV cache

`T2sState` holds, per layer, the keys and values computed so far, plus an `offset` counting positions consumed. `FusedAttention::forward` concatenates the new step's keys and values onto the cache and attends over the union, so a step costs one position of work rather than re-running the whole prefix.

Two implementation details are called out in the source because they load fine when wrong. The attention projections are **fused**: `nn.MultiheadAttention` stores Q , K and V stacked in one `in_proj_weight` of $3d$ rows, so one matmul produces all three and they are sliced apart. And the layers are **post**-norm with no final norm — the checkpoint has no `h.norm`.

The test that earns its keep here is `incremental_generation_matches_one_shot`: feeding a sequence whole and feeding it token by token must agree at the final position. That is true only if the causal mask **and** the positional offset are both right under a growing cache, and neither is visible to weight coverage.

4.4 Choosing the next token

`sample` turns logits into an id. Greedy `argmax` is not an option: over a thousand near-equivalent codebook entries it collapses into repeating one sound. Four knobs shape the draw, applied in this order:

Repetition penalty (1.35) every token already generated has its score pushed **down**. Note the asymmetry — a positive score is **divided** by the penalty and a negative one **multiplied**, so both move down. Applying one rule to both would **raise** the negative case and make repetition more likely, the exact opposite of the intent. It is applied **before** the cuts, so a penalised token can drop out of the top- k entirely. This is the mechanism that stops a run of one token from becoming self-reinforcing: without it, a repeated token keeps winning, the utterance never reaches EOS, and generation runs to the token cap.

Temperature divides all scores. Below 1 sharpens, above 1 flattens.

Top- k (15) keep only the 15 highest-scoring ids, and softmax over those survivors alone.

Top- p (nucleus, off by default) keep the shortest prefix of that ordering whose probability sums past p .

The generator is a small seeded xorshift (`Rng`), which is what makes a synthesis exactly reproducible from `SynthOptions::seed`.

5 s2 — a conditional VAE around a vocoder

`SovitsPartial` is a modified VITS, and most of it is literally `burn-vits` — the attention stack, the WaveNet, the flow, the posterior encoder, `ResBlock1`, the weight-normalised convolutions, the discriminators and the losses are shared with RVC, because both projects descend from the same source and their `state_dict` names line up. What is GPT-SoVITS's own is `enc_p` (with its MRTE), `ref_enc`, and the quantiser.

As a **conditional variational autoencoder** it has the familiar VITS shape: two encoders that each emit a per-frame distribution over a 192-dimensional latent, an invertible flow bridging them, and a decoder that renders the latent.

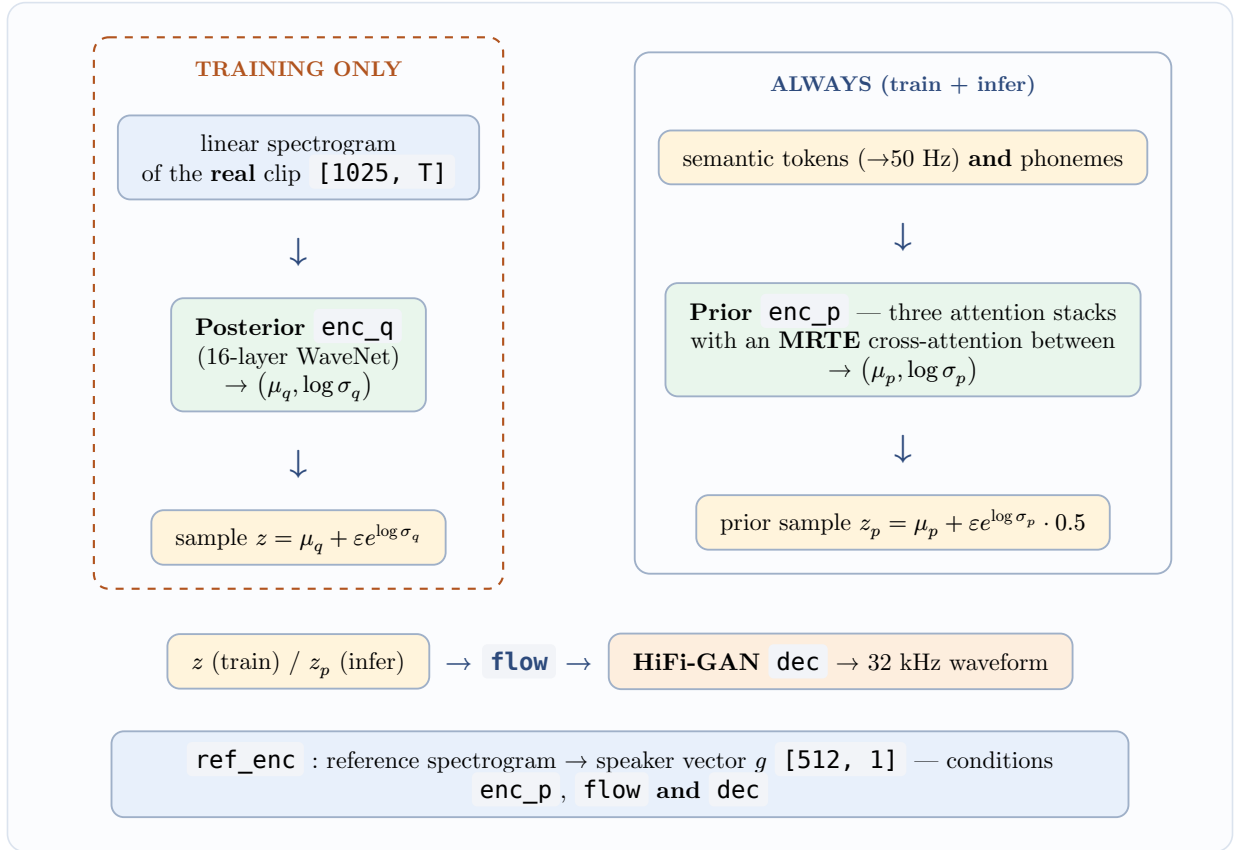


Figure 4: The five modules and the two data paths. The posterior branch (dashed) exists **only during training** — it is the cheat sheet that teaches the prior what a decodable latent looks like. The speaker vector g conditions everything on the right.

5.1 The prior: `enc_p` and the MRTE

`TextEncoder` (`enc_p`) is where the two input streams meet. It is three attention stacks and one cross-attention:

1. `ssl_proj` narrows the 768-wide quantised features to 192, and `encoder_ssl` (3 layers) runs over them — the **semantic** side, one position per 50 Hz frame.
2. `text_embedding` looks up the phoneme ids in a 732×192 table, and `encoder_text` (6 layers) runs over them — the **text** side, one position per phoneme.
3. `mrte` lets the semantic side attend over the text side.
4. `encoder2` (3 layers) runs over the result, and `proj` emits 2×192 channels, split into μ_p and $\log \sigma_p$.

The **MRTE** — multi-reference timbre encoder — is the one mechanism here that is GPT-SoVITS’s own, and it answers a real question: `s1` already turned the phonemes into tokens, so why show `s2` the phonemes again?

Because the tokens are lossy. They are a 1024-way quantisation at 25 Hz, and a token sequence is compatible with a range of pronunciations. The phonemes say unambiguously **which** sounds were meant. `Mrte::forward` widens both sides to 512 channels, runs a cross-attention with **queries from the semantic side and keys and values from the text side**, adds the speaker vector, and narrows back to 192:

$$y \leftarrow \text{c_post}(\text{CrossAttn}(\text{c_pre}(y), \text{text_pre}(x_{\text{text}})) + \text{c_pre}(y) + g).$$

Two properties follow from that shape and both are pinned by tests. The residual is on the **semantic** side, so the output has one vector per **semantic frame** however many phonemes came in — get it

backwards and the decoder produces audio the length of the transcript rather than of the utterance. And because query and key come from different sequences, `CrossAttention` has **no relative-position embeddings**: the checkpoint has `conv_q/k/v/o` and no `emb_rel_k / emb_rel_v`, and rightly so, since the distance between an index in one sequence and an index in another means nothing.

5.2 The posterior and the flow

`enc_q` (`PosteriorEncoder`, shared with RVC) sees the **real answer** — the 1025-bin linear spectrogram of the ground-truth waveform — through 16 dilated WaveNet layers, and emits $(\mu_q, \log \sigma_q)$. Its latent is easy for the decoder to render because it was derived from the audio it must reproduce. At inference there is no ground-truth audio, so `enc_q` is unavailable; it exists to **teach**.

The bridge is a **normalising flow**: `ResidualCouplingBlock`, four additive coupling layers with a channel flip between each. A layer splits the 192 channels in half, leaves the first half untouched, and adds to the second half a WaveNet-predicted shift computed **from the first half**:

$$x_1 \leftarrow x_1 + m(x_0, g) \quad (\text{forward}), \quad x_1 \leftarrow x_1 - m(x_0, g) \quad (\text{reverse}).$$

Because the shift depends only on the untouched half, inversion is exact — reverse subtracts the very same predicted mean. The layers are **mean_only**, so the log-determinant is zero and there is nothing extra to track. The flip between layers ensures every channel is eventually transformed. GPT-SoVITS’s couplings are 4 WaveNet layers deep where RVC’s are 3; that is the only difference, and it is a constructor parameter rather than a fork.

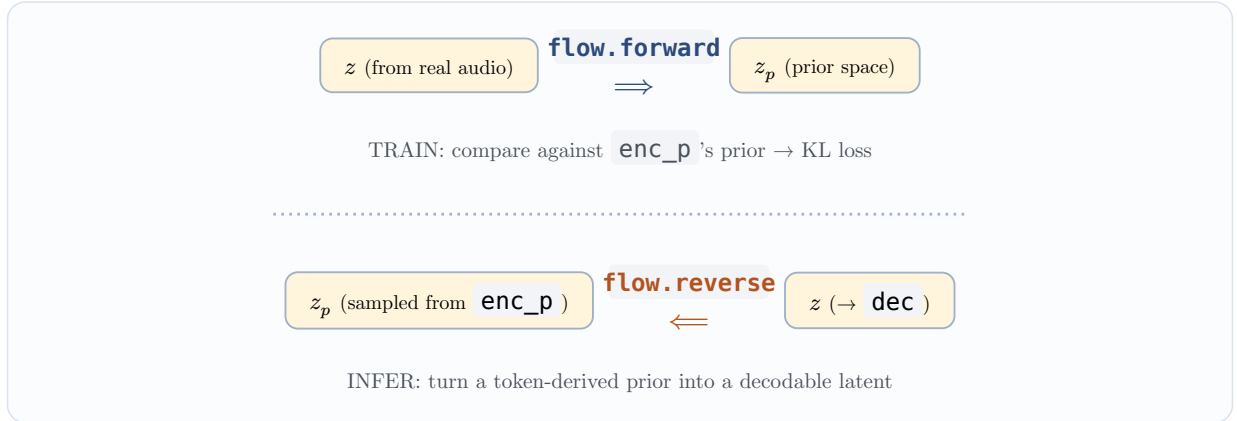


Figure 5: One function, two directions. Training pushes the posterior latent into the prior’s space so the KL term can compare them; inference pulls a prior sample into the space the decoder was trained on.

At inference the prior is sampled **cooled**: `SovitsPartial::decode` takes a `noise_scale`, upstream’s default 0.5, so $z_p = \mu_p + \epsilon e^{\log \sigma_p} \cdot 0.5$ — deliberately less variance than the distribution suggests, trading diversity for stability.

5.3 The speaker vector: `ref_enc`

This is where a synthesised voice gets its timbre, and it is the sharpest architectural difference from RVC. RVC looks its speaker up in a table of **trained ids**, so it can only produce voices it was trained on. GPT-SoVITS **computes** its speaker vector from a reference recording, which is what makes few-shot cloning possible at all.

`ReferenceEncoder` is a MelStyleEncoder: two pointwise layers under a `mish` activation, two gated convolutions (`Conv1dGlu` — the convolution emits twice the channels, half signal and half a sigmoid gate), one round of self-attention, and then an **average over time**. The average is the point: three

seconds of reference and thirty produce the same $[512, 1]$ shape, because a speaker is a property of the whole clip rather than of any frame in it. The trailing axis of length one is deliberate — everything downstream broadcasts g across time.

Two constants here are the kind that load perfectly and compute the wrong thing. `ReferenceConfig::in_dim` is **704**, a hardcoded upstream constant rather than any mel count, so `SovitsPartial::speaker` slices the first 704 spectrogram bins. And `StyleAttention` scales by $\sqrt{d_{\text{model}}}$, **not** $\sqrt{d_{\text{head}}}$ — upstream passes `temperature = d_model ** 0.5`, so with two heads over 128 channels that is $\sqrt{128}$ where the conventional choice is $\sqrt{64}$. The conventional choice is wrong here by a factor of $\sqrt{2}$.

5.4 The vocoder: `dec`

`Decoder` is plain HiFi-GAN — `burn-rvc`’s generator **without** the NSF source module, because GPT-SoVITS does not condition on an F_0 contour. It takes the 192-channel latent and upsamples through five transposed convolutions with rates $10 \cdot 8 \cdot 2 \cdot 2 \cdot 2 = 640$, halving the channel width at each stage from 512. After each upsample the output is passed through three `ResBlock1` blocks with kernels 3, 7 and 11 (dilations $1 \cdot 3 \cdot 5$) and their outputs are **averaged**. A final `tanh` bounds the result to $[-1, 1]$, which is what makes it a waveform rather than an unnormalised signal.

The speaker vector is added once, by `cond`, **before** any upsampling — it colours the whole utterance rather than varying along it.

That 640 is not a free parameter: it is the hop of the 32 kHz spectrogram the rest of `s2` works in (`SpectralConfig::gptsovits_v2_32k`), so one latent frame is one spectrogram frame is 20 ms of audio, and the decoder’s output lines up with the posterior encoder’s input sample for sample. The two ends of the model are trained against each other, so they must agree exactly.

5.5 A note on dimensions (v2, 32 kHz)

quantity	value	meaning
codebook	1024×768	<code>Codebook::embed</code> ; ids are the semantic vocabulary
semantic rate	25 Hz	cnhubert’s 50 Hz halved by <code>ssl_proj</code>
<code>s1</code> transformer	24×512 , 16 heads	<code>T2s</code> , post-norm, FFN 2048
<code>s1</code> vocabularies	732 / 1025	phonemes / codebook + EOS
prosody width	1024	<code>chinese-roberta-wwm-ext-large</code> , narrowed by <code>bert_proj</code>
<code>spec_channels</code>	1025	linear-spectrogram bins ($n_{\text{fft}}/2 + 1$) into <code>enc_q</code>
<code>inter_channels</code>	192	latent width shared by <code>enc_p</code> , <code>enc_q</code> , <code>flow</code> , <code>dec</code>
<code>gin_channels</code>	512	speaker-vector width g from <code>ref_enc</code>
<code>enc_p</code> stacks	3 / 6 / 3	<code>encoder_ssl</code> / <code>encoder_text</code> / <code>encoder2</code>
MRTE	512 wide, 4 heads	cross-attention, no relative positions
<code>ref_enc</code> input	704	hardcoded upstream slice, not a mel count
flow	4 couplings $\times$ 4	additive, <code>mean_only</code> , with flips
upsample rates	$10 \cdot 8 \cdot 2 \cdot 2 \cdot 2$	product = hop = 640 samples/frame
output	32 kHz	$32000/640 = 50$ latent frames per second

6 Prosody: the frozen Chinese BERT

Phonemes say which sounds occur; they say nothing about which words carry emphasis, where a clause ends, or whether a sentence is a question. GPT-SoVITS recovers some of that from **text semantics**, by conditioning `s1` on features from `chinese-roberta-wm-ext-large`.

Three details make it work, and each is a silent failure if missed:

The third-from-last hidden layer, not the last upstream reads `hidden_states[-3]`. A stock `optimum` export gives `last_hidden_state`, which is a different representation — the model would sound wrong rather than fail.

word2ph alignment BERT produces one feature per **character**, while `s1` wants one per **phoneme**. `text-kit` reports, for each character of the normalised text, how many phonemes it produced; `OnnxProsody::encode` repeats each character's row that many times. The `[CLS]` and `[SEP]` tokens are stripped first, and a mismatch between the character count and `word2ph` is a hard error (`TtsError::Misaligned`) rather than a silent truncation.

Zeros are a legitimate input the encoder is Chinese-only, and upstream feeds all-zero features for other languages. `ProsodyFeatures::zeros` is that value, not a fallback dressed up as one. Losing prosody costs **expressiveness**, not **intelligibility** — synthesis without it still speaks, just flatter.

Why this one is ONNX. Every other network in the toolkit is a Burn port, and this one is deliberately not. The rule is not loyalty to Burn but whether the model is **trained here**. A model that is fine-tuned must be a Burn port, because there is no ONNX training path — that is why `s1` and `s2` are ports. This BERT is **frozen**: GPT-SoVITS never touches it. And it is small work in the wrong place — one sentence is a few dozen tokens through 24 layers, so the run is dominated by kernel-launch overhead rather than arithmetic, and a port would not be meaningfully faster. The autoregressive loop downstream, which runs hundreds of steps per utterance, is where speed is worth chasing. `ProsodyEncoder` is a trait, so the slot stays open — the same arrangement `stt-core` uses for its two runtimes.

7 The phoneme table is a compatibility contract

`text_kit::symbols::SYMBOLS` is GPT-SoVITS's v2 vocabulary verbatim: 732 entries, in upstream's exact order. This is not a stylistic choice about where to keep a list. **Those indices address the model's phoneme embedding.** An off-by-one table does not raise an error; it shifts every phoneme to its neighbour and the model produces confident nonsense.

The order is also not derivable from anything tidy. Upstream sorts a union of per-language symbol sets and then **appends two more groups unsorted**, so the table is reproduced as data rather than as that construction. The same reasoning makes `opencpop-strict.txt` an `include_str!` rather than a runtime read: a file that can go missing is a contract that can be broken at the user's machine instead of at compile time.

The nice part is that the claim is independently checkable. In a real checkpoint, `s2G2333k.pth`, the tensor `enc_p.text_embedding` has shape `[732, 192]` — and 732 is exactly `SYMBOLS.len()`. Two files written by different people, in different languages, agreeing on a number that neither derives from the other.

Unknown symbols map to `UNK` rather than to index 0, because index 0 is a real symbol and mapping to it would feed a plausible-looking wrong answer into the model instead of an obvious one.

8 Training I: `s1`, one loss that means something

Fine-tuning `s1` is plain next-token prediction, and after a GAN it is a relief: **one model, one optimizer, one loss**. The number on the dashboard is an honest measure of fit, so it can be read

the way a loss is normally read — down is better, and a plateau means what it looks like. (Contrast §9, where two networks are adversaries and neither loss descends to zero by design.)

A corpus is `<stem>.wav` beside `<stem>.txt`, and `stt` is how the transcripts get written. Preparation runs the two frozen encoders over it once — cnhubert plus the quantiser turn each clip into tokens, the prosody BERT turns each transcript into per-phoneme features — because neither changes during training. The encoders are then **dropped** before training starts, which matters on a small card where they would otherwise sit beside the model being trained.

Each step runs `T2s::forward_prompt_all`, which differs from the inference forward in one respect: it returns logits for **every** audio position rather than just the last. Inference wants only the last, because it generates one token at a time; training wants all of them, because each position predicts the next and so one pass over a clip supplies as many examples as it has tokens. The text positions are dropped — nothing predicts a phoneme.

The target is the token sequence shifted by one with `EOS` appended:

$$L_{s1} = -\frac{1}{N} \sum_{t=1}^N \log p(c_t \mid c_{<t}, P), \quad c_N = \text{EOS},$$

where P is the phoneme-and-prosody prompt. Appending `EOS` is what teaches the model to **stop**; it is the entire reason the token is in the vocabulary.

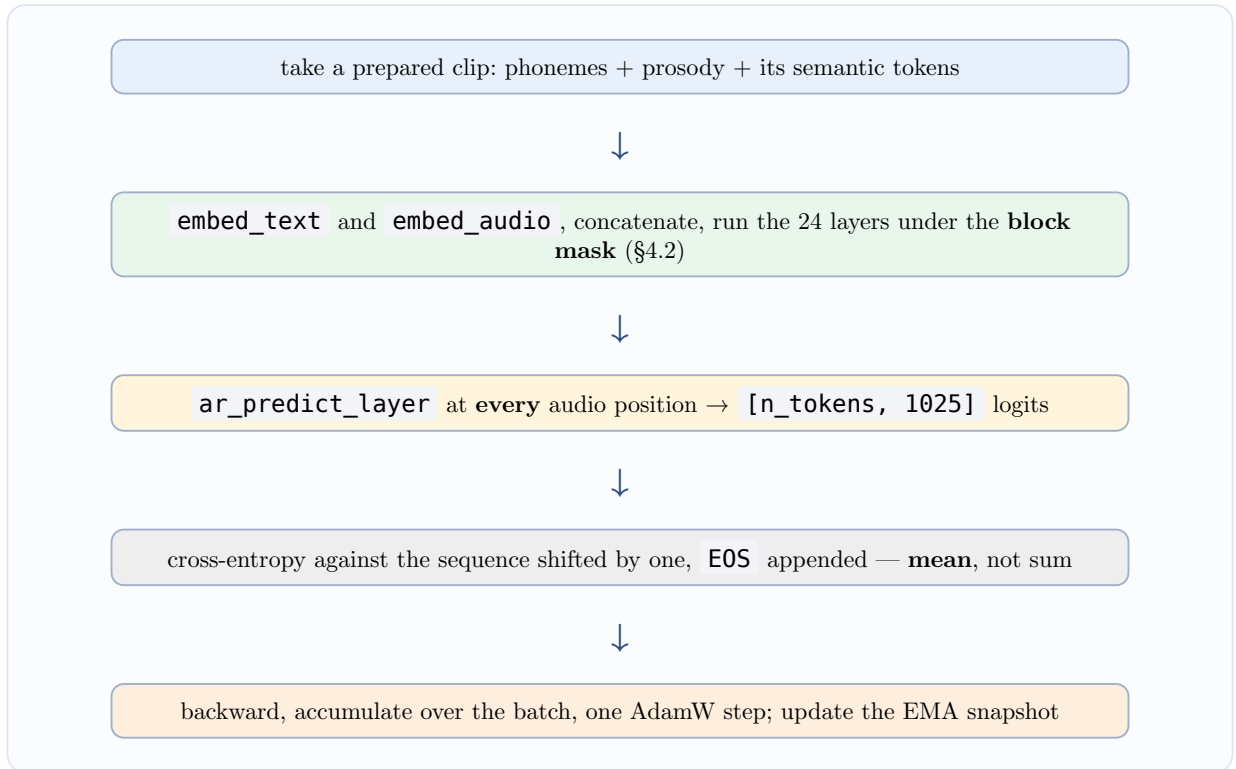


Figure 6: One `s1` fine-tuning step. No adversary, no second optimizer, and the loss is comparable across runs.

Three choices in `tts-train`’s loop are worth recording because each has a reason that is not obvious:

- **Mean cross-entropy, not upstream’s sum.** A sum makes the gradient scale with clip length, so long clips dominate a corpus of mixed lengths and the learning rate means something different for every one.
- **Clips over `--max-tokens` are skipped, not truncated.** A cut-off sequence has no `EOS` where the model is taught one, so truncation teaches it to stop early.

- **A batch is accumulated, not padded.** Clips differ in length and this loop does not pad, so `batch_size` clips are processed one at a time and their gradients summed — the honest default is 1.

The learning rate decays exponentially across the whole run ($\text{lr}(t) = \text{lr}_0 \cdot \rho^{t/T}$, `--lr-final` being ρ), and what is saved is an **exponential moving average** of the weights rather than the last step — both the same shape as `rvc-train`, and both for the same reason: a small corpus at a flat rate overfits fast and the final step is a noisy place to stop.

9 Training II: `s2`, the VITS adversarial recipe

`s2` teaches **timbre**, and it is trained the way every VITS descendant is: a compound reconstruction loss plus a GAN. Few-shot cloning from a reference clip already works without this — fine-tuning `s2` is for when a particular voice is worth more than a few seconds of prompt.

Status. This section describes the **objective** — what an `s2` fine-tune has to compute and why. The `s1` loop ships in `tts-train`; the `s2` loop is being written. The building blocks it needs are all present and verified: `burn-vits` supplies `mel_l1`, `kl`, `gen_adv`, `disc_loss`, `feature_matching`, the differentiable `Spectral` front-end and `MultiPeriodDiscriminator`; `train-kit` supplies `Checkpoint`, `ema_update`, `accumulate`, `materialize` and the `Dashboard`; and `rvc-train` is a working reference for the same recipe on a sibling model.

9.1 The training forward

The training path differs from inference in three places, and it is easier to see them side by side than to describe them:

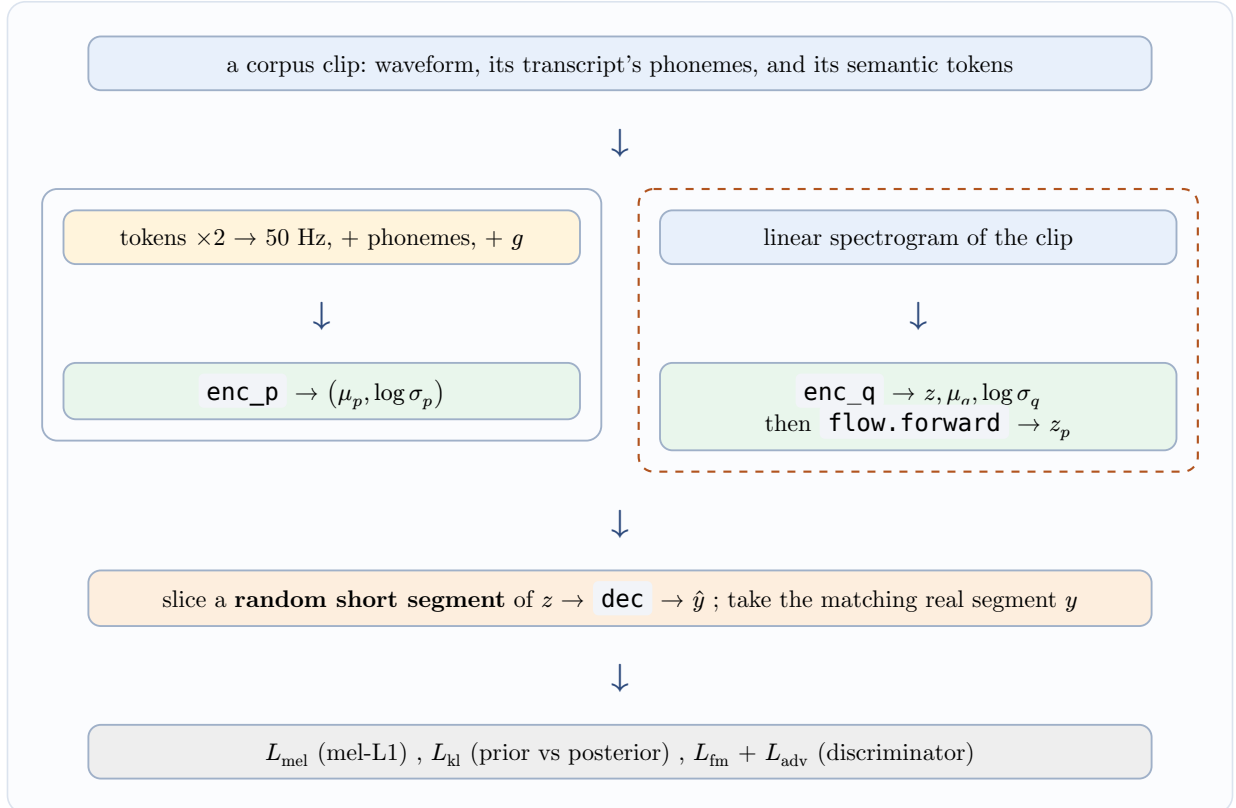


Figure 7: The `s2` training forward. Green is what inference also runs; orange dashed is training-only. The speaker vector comes from the clip's **own** spectrogram — at training time the reference and the target are the same recording.

Note what conditions what. `ref_enc` reads the clip’s **own** spectrogram, so at training time reference and target coincide and g is simply “this speaker”. Only a random segment is decoded — the vocoder is the expensive part and it does not need the whole clip to learn. And the quantiser is normally **frozen**: its codebook defines the interface `s1` was trained against, so moving it would invalidate the other stage.

9.2 The five terms

Writing D_k for the k -th discriminator’s score and $D_k^{(l)}$ for its l -th intermediate feature map:

Mel-spectrogram L1 ($\times 45$) the perceptual backbone, and what actually makes the output **sound** like the target. Its weight reflects that.

$$L_{\text{mel}} = \|\text{mel}(y) - \text{mel}(\hat{y})\|_1.$$

KL divergence ($\times 1$) the CVAE glue. It pushes the flow-transformed posterior z_p towards `enc_p`’s prior, so that at inference — when only the prior exists — the decoder still receives a latent it knows how to render. VITS’s Monte-Carlo form, on the sampled z_p rather than in closed form:

$$L_{\text{kl}} = \log \sigma_p - \log \sigma_q - \frac{1}{2} + \frac{1}{2}(z_p - \mu_p)^2 e^{-2 \log \sigma_p}.$$

Feature matching ($\times 2$) match the generated audio to the real **inside** the discriminator, layer by layer — a learned perceptual L_1 that stabilises the adversarial game. The real features are **detach**ed; they are fixed targets.

$$L_{\text{fm}} = \sum_k \sum_l \left\| D_k^{(l)}(y) - D_k^{(l)}(\hat{y}) \right\|_1.$$

Generator adversarial (LSGAN, $\times 1$) push the discriminator’s verdict on generated audio towards 1.

$$L_{\text{adv}(G)} = \sum_k \mathbb{E}[(D_k(\hat{y}) - 1)^2].$$

Discriminator (LSGAN) push real towards 1 and generated towards 0, on **detached** generated audio so its gradient never leaks into the generator.

$$L(D) = \sum_k \mathbb{E}[(D_k(y) - 1)^2] + \mathbb{E}[D_k(\hat{y})^2].$$

The complete generator objective is the weighted sum

$$L_G = L_{\text{adv}(G)} + 2L_{\text{fm}} + 45L_{\text{mel}} + L_{\text{kl}},$$

with one addition when the quantiser is **not** frozen: the VQ **commitment** term, which pulls the encoder’s continuous output towards the codebook entry it was assigned, keeping the two from drifting apart. Freezing the quantiser removes it.

The least-squares (LSGAN) formulation is used rather than the log-loss original because it is markedly more stable — a saturated log-loss discriminator gives the generator no gradient at all.

9.3 Multi-period plus multi-scale

A single discriminator reading the raw 1-D waveform misses **periodic** artefacts — the buzz and roughness that live in the pitch harmonics. `MultiPeriodDiscriminator` is therefore a bank:

- One **scale** discriminator (`DiscriminatorS`) reads the waveform as a plain 1-D signal, good at broad envelope and noise texture.
- Five **period** discriminators (`DiscriminatorP`), one per period in $\{2, 3, 5, 7, 11\}$ for GPT-SoVITS v2. Each **folds** the 1-D waveform into a 2-D grid with that period as the row stride and runs 2-D convolutions over it. Folding on a period lines up samples that spacing apart, exposing exactly the

structure a periodic artefact lives in. Coprime periods cover a wide range of periodicities without redundant overlap.

The period list is the only thing that differs across the family — RVC v2 uses eight, GPT-SoVITS v2 five, v2Pro seven — so it is a constructor parameter, and the checkpoint remap is generated from its length rather than written out per model. Both families also expose their intermediate feature maps, which is what L_{fm} consumes.

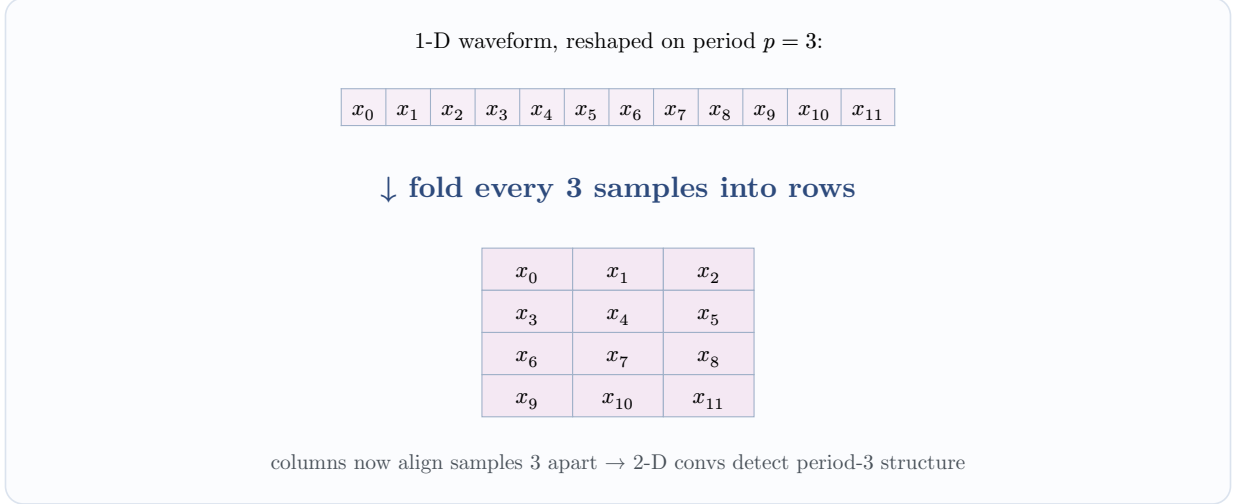


Figure 8: How a period discriminator sees the signal: a period-3 fold turns a 1-D strip into rows whose columns line up samples 3 apart, then convolves in 2-D.

One porting trap is baked into `DiscriminatorS::forward` and must not be removed casually. burn 0.21’s autodiff builds a wrongly-shaped weight gradient for a **grouped, strided conv1d** whose padded length is not a multiple of the stride; CubeCL and WebGPU absorb it, LibTorch aborts. `DiscriminatorS` is exactly that shape — four $k\{=41, s\{=4\}$ layers, so it needs $\text{len} \bmod 256 = 1$ — and it reflect-pads its input up to `SCALE_ALIGN` to satisfy the chain. That costs under 1% of a segment, changes no weight shapes, and is what lets `--backend tch` train at all.

9.4 Reading the loss curves

Because G and D are adversaries their losses do **not** both fall to zero; they seek an equilibrium. The practical reading is the same as `rvc-train`’s: `mel_loss` is the honest reconstruction signal and should trend down, with some shaking around a plateau in the back half being normal; a `d_loss` collapsing to 0 means the discriminator has won and the generator is getting no useful gradient; and `g_loss`, dominated by the $45 \times \text{mel}$ term, largely tracks `mel_loss` plus adversarial jitter.

10 Verifying a port beyond weight coverage

This is the section to read before changing anything in `burn-gptsovits`.

The first check is **weight coverage**: `examples/load` builds a component, applies a real published checkpoint, and reports applied / missing / unused.

component	applied / missing	note
hubert	210 / 0	chinese-hubert-base/pytorch_model.bin
quantizer	3 / 0	the codebook slice of an s2G*.pth
sovits	773 / 0	3 unused: the codebook’s EMA training statistics
t2s	295 / 0	an s1*.ckpt

That number says the module **tree** matches the checkpoint. It says nothing whatsoever about whether the forward pass computes the right thing, and this repository has already shipped a port that loaded at 100% and produced garbage — Whisper’s reversed causal mask. Several traps documented above are of exactly that kind: post-norm where the reference is pre-norm, **StyleAttention**’s $\sqrt{d_{\text{model}}}$, an MRTE wired with its residual on the wrong side. **All of them load at 100%.**

So each model needs a second check that exercises arithmetic. There are two, and they are cheap:

examples/reconstruct — **s2** end to end, no reference implementation needed real audio $\rightarrow$ cnhubert $\rightarrow$ quantise $\rightarrow$ **SovitsPartial::decode**, using the input’s own spectrogram as the speaker reference. Then **correlate the output’s energy envelope against the input’s**. A correct port tracks it at $r = 0.91$ where shuffling gives a chance baseline of 0.30; spectral flatness comes out at 0.17 against 1.0 for noise. This is not a fair test of **quality** — the phonemes are arbitrary and the prior is sampled — but a mis-wired MRTE or a mis-scaled attention returns noise, and speech is unmistakable against noise.

tts then stt, the **end-to-end oracle** synthesise a line, transcribe the result with the speech-recognition engine, and compare the text that went in with the text that came out. An independent model listening to the output is as close to a human check as an automated one gets, and it exercises **text-kit**, **s1**, **s2** and the prosody encoder in one shot.

Two smaller habits complete the picture. **examples/keys** lists any checkpoint’s tensor names and is the first thing to run against an unfamiliar one — it is how the remaps in **SovitsPartial::load_pytorch** were derived. And the unit tests in each module deliberately target the properties that coverage cannot see: that the stride halves the frame rate, that a code survives an encode/decode round trip, that the flow preserves its input shape, that incremental generation matches one-shot, that any length of reference gives one speaker vector.

A closing note on refactoring, because it removes a fear that would otherwise make this code rigid: **moving a module between crates is free**. Burn derives parameter paths from the field names of the struct that **contains** a module, not from the crate it was declared in. That is what made **burn-vits** extractable with every checkpoint untouched. Renaming a **field** does move a path; renaming a **type** or moving a **file** does not.

11 Inference recap: the reverse path

At synthesis time the posterior encoder and the whole discriminator bank are gone. What remains is:

1. Analyse the reference clip **once**: cnhubert plus the quantiser give the prompt tokens; the 32 kHz spectrogram through **ref_enc** gives the speaker vector g .
2. Phonemize the reference transcript **and** the target line, and run both through the prosody encoder.
3. Run **s1**: one pass over the whole prompt under the block mask, then one token at a time against a growing KV cache, sampling with top- k and the repetition penalty, until EOS or the token cap.
4. Run **s2**: the generated tokens (repeated $\times 2$ to 50 Hz) and the target’s phonemes through **enc_p**, sample the prior cooled by **noise_scale**, **reverse** the flow, and decode with g .

The asymmetry is the whole idea. **What is said and how it is paced** came from the text, through a language model over a discrete vocabulary. **What it sounds like** came from a few seconds of reference audio, through an average-pooled speaker vector. Neither half had to learn the other’s job, and the 25 Hz codebook in the middle is what let them be separated.

Concepts anchored to the **voice** implementation: **burn-gptsovits** (the network), **burn-vits** (the shared VITS blocks and losses), **text-kit** (g2p and the 732-symbol table), **tts-core** (synthesis), **tts-train** (fine-tuning).
GPT-SoVITS v2, 32 kHz, weight-compatible with the published checkpoints.